CSE412 – Group Project

Phase 1: Proposal

by

Team 19

Aarush Murari
Yi-Chen Zeng
Nathan Feig
William Zenko

1. Detailed Application and Agent Requirements

Proposed Application

Our application follows the client-server architecture with a JavaScript/TypeScript front-end communicating with a Python FastAPI backend.

The dataset we chose to use is the CIMS Two-Stage Cascade Mini-Fab semiconductor simulation dataset, which consists of 18 experiment configurations with a CSV for each machine, coordinator, transductor and time-series.

Our application will use Python for the backend, the Gemini API for the LLM agent, and PostgreSQL as the primary database system. The LLM agent will act as an intelligent planner and natural language interface. PostgreSQL will remain responsible for storing, querying, aggregating, and processing the semiconductor manufacturing simulation data.

Major Data Objects

The major entities represented in our database include the entities shown in the ER diagram:

- Experiment
- Trajectory
- Observation
- Component
- Stage
- Machine
- Segment
- Transducer

Data Sources

The dataset we chose to use is the CIMS Two-Stage Cascade Mini-Fab semiconductor simulation dataset, which consists of 18 experiment configurations with a CSV for each machine, coordinator, transductor and time-series.

User and Data Interactions

When a user submits a natural language request, the Gemini agent will interpret the request and select the appropriate backend tool. The Python backend will execute the selected tool and communicate with PostgreSQL when necessary. The query result will then be returned to Gemini, which will generate the final natural language response for the user.

The LLM cannot query Postgres directly and needs to use these tools to inspect the database.

Database Operations

PostgreSQL will remain responsible for storing, querying, aggregating, and processing the semiconductor manufacturing simulation data.

The execute_sql(query) tool supports operations such as filtering, joins, grouping, aggregation, and sorting. The LLM will only have read-only access to the database through a SELECT-only validation process, and a read-only postgres role.

Natural Language User Requests

The completed agent should be able to handle requests such as:

- "Show me the available information for experiment E001."
- "Which machines are associated with experiment E001?"
- "Show me the observations for a particular trajectory and order them by logical time."
- "Compare the machine performance between two experiments."
- "Which machines are the bottlenecks in experiment E001 based on throughput?"
- "Which machines are the bottlenecks in experiment E001 based on turnaround time?"
- "What are the average observed values for a particular trajectory?"

2. ER Diagram

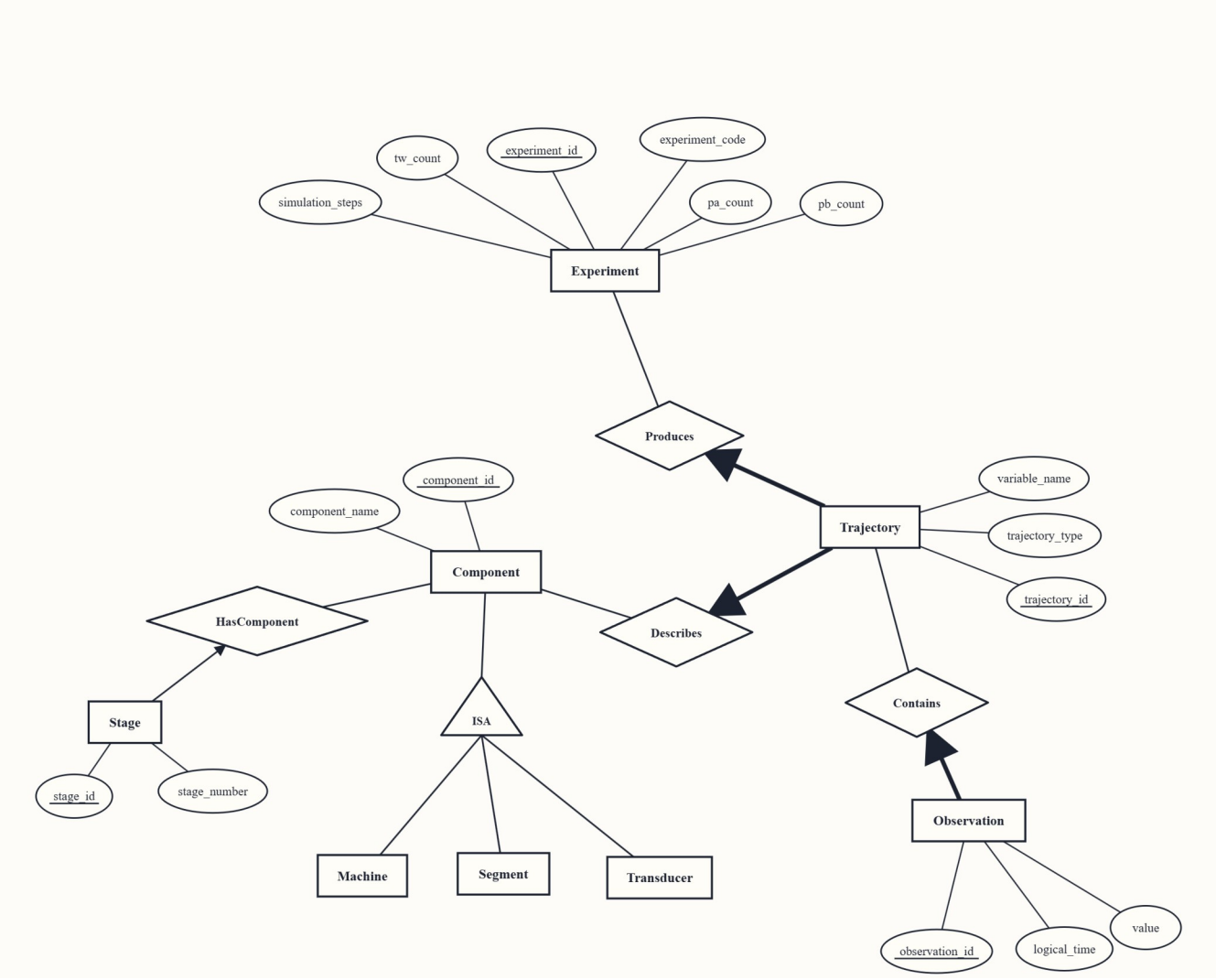


Figure 1. Entity-Relationship Diagram

The ER diagram models the entities, attributes, relationships, keys, and cardinality constraints required by the application.

3. LLM Agent and Tool Design

Agent Architecture

Our agent follows the standard tool-calling loop:

User Request → LLM Agent → Tool Selection → Tool Execution → PostgreSQL → Result → LLM Agent → User.

All database access is mediated through the three tools below, and the LLM can only query postgres through one of them.

Agent Tools

Tool	Input	Output	Purpose
get_database_schema()	Optional table name	Schema metadata	Inspect database structure
execute_sql(query)	SQL query	Query results	Execute read only database queries
get_bottleneck_machines(experiment_id, metric)	Experiment ID and metric	JSON	—

get_database_schema():

This tool will take no inputs and will return every table and their attributes (columns) for the LLM to understand — most likely in a JSON document.

This tool retrieves the PostgreSQL database schema, including available tables, columns, data types, primary keys, and relationships. The Gemini agent can use this information before generating SQL so that it does not reference nonexistent tables or attributes.

execute_sql(query):

This will allow the LLM to directly execute SQL queries on the database. The LLM will only have read-only access to the database through a SELECT-only validation process, and a read-only postgres role.

Just like the previous tool, the output for this tool will be a JSON object.

This tool executes read only SQL queries against PostgreSQL and returns the query results to the Gemini agent. It supports operations such as filtering, joins, grouping, aggregation, and sorting. Database modification commands will not be allowed through this tool.

get_bottleneck_machines(experiment_id, metric):

Takes a string experiment_id and a metric argument restricted to either "turnaround" or "throughput". Rather than having the LLM write SQL for this common query, it uses a pre-built query that minimizes its chances of error.

The tool’s output is also going to be JSON.

4. Implementation Plan

The backend will be implemented in Python, with PostgreSQL serving as the primary database management system. The provided semiconductor manufacturing CSV files will be cleaned, transformed, and imported into relational tables in PostgreSQL. The Gemini API will be used to implement the LLM agent and tool calling functionality.

The application follows the client-server architecture with a JavaScript/TypeScript front-end communicating with a Python FastAPI backend. The backend hosts all the agent logic which will use Gemini free-tier with an Open-AI compatible API with native function calling to invoke our three tools — get_database_schema(), execute_sql(query), and get_bottleneck_machines(experiment_id, metric) — against a PostgreSQL database. The LLM cannot query Postgres directly and needs to use these tools to inspect the database. The backend runs each tool to fetch the data from the database and return them to the LLM as a JSON document.

Our application will use Python for the backend, the Gemini API for the LLM agent, and PostgreSQL as the primary database system. The LLM agent will act as an intelligent planner and natural language interface. PostgreSQL will remain responsible for storing, querying, aggregating, and processing the semiconductor manufacturing simulation data.

When a user submits a natural language question, Gemini will interpret the request and determine which backend tool should be invoked. The Python backend will execute the requested tool and communicate with PostgreSQL when necessary. The database result will then be returned to Gemini, which will generate the final natural language response.

A simple Streamlit interface or command line interface may be used for user interaction.

5. Proposal Presentation

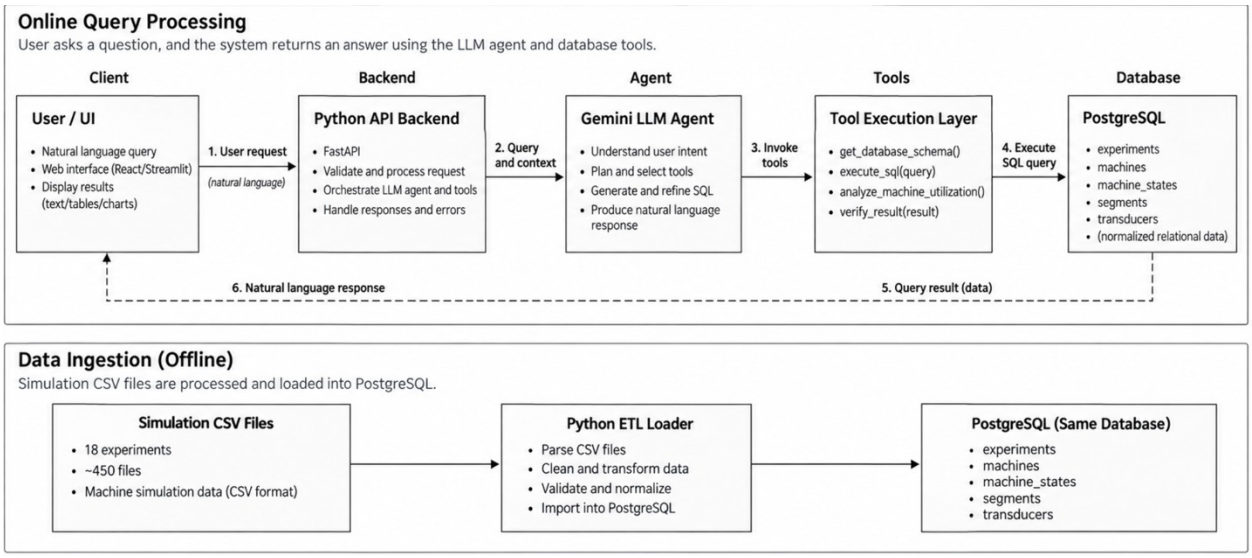


Figure 2. LLM Agent and Backend Architecture

The 5-minute video presentation will introduce:

- The proposed application;
- Data sources;
- ER diagram;
- Example user requests; and
- Proposed LLM agent and tools.

YouTube Video Demo
https://youtu.be/wP4reb4eTeY